

SOLUTION APPROACH

PARALLAX

An AI investigator for fraudulent Android applications: generative-AI reverse engineering, static and dynamic analysis, and evidence-first risk scoring.

CHALLENGE	Harnessing Generative AI for Automated Reverse Engineering, Static and Dynamic Analysis, and Risk Scoring of Fraudulent Mobile Applications (APKs) and Malware
CONTENTS	Solution approach, system architecture, datasets, open-source stack, validation
STATUS	Working prototype, validated end-to-end on a live banking-trojan sample

Executive Summary

PARALLAX treats every submitted APK as a case to investigate, not a file to scan. A classical mobile-malware analysis stack (decompilation, taint analysis, an instrumented Android emulator) feeds a multi-agent generative-AI reasoning layer that interprets the evidence, attributes the malware family, and produces an explainable risk verdict. Every analysis also enriches a shared knowledge graph, so detection compounds with volume rather than resetting on each sample.

The design answers the three properties that defeat signature-based defenses: heavy runtime obfuscation, malicious surfaces hidden behind accessibility and notification services, and hourly polymorphic repackaging. The verdict is produced by a deterministic, auditable scoring model over evidence items; the language model writes the narrative, it does not decide the score.

5 pipeline stages	7 reasoning agents	~\$0.16 LLM cost / APK	6 outputs per analysis	20+ open-source tools
-----------------------------	------------------------------	----------------------------------	----------------------------------	---------------------------------

1 Problem Context

Fraudsters distribute malicious Android applications through WhatsApp, SMS, email, and phishing links, then use them to steal customer credentials, harvest sensitive data, and authorize fraudulent transactions against bank accounts. The analysis that stops them is still largely manual: slow, costly, and dependent on a small pool of expert reverse engineers. Sample volume and velocity have outgrown that model.

The technical reasons signature-based and manual defenses fall behind are threefold. Current banking-trojan families such as Cerberus, Hydra, SharkBot, Anatsa, SpyNote, and TeaBot share three properties:

- **Heavy runtime obfuscation.** R8 and DexGuard, string encryption, dead-code injection, and control-flow flattening blunt static signatures.
- **Gated malicious surface.** The payload lives in an AccessibilityService or NotificationListenerService, not the launcher activity. The manifest main activity is a decoy, so naive sandboxing that spawns the launcher never reaches the payload.
- **Polymorphic repackaging.** The same family is re-signed and re-bundled hourly by malware-as-a-service operators, which structurally outmatches hash-based detection.

A pipeline that ships a fresh signature within 24 hours of a new sample is already a day behind. The challenge calls for generative AI applied to the analysis itself: not a faster signature matcher, but a system that reasons about the APK as an investigator would, forms hypotheses, and tests them at runtime.

2 Solution Approach and Architecture

PARALLAX is a five-stage pipeline with a cross-cutting knowledge layer. Each stage is independently callable, and the reasoning layer is idempotent, so it can be re-run as new evidence arrives without restarting the analysis.

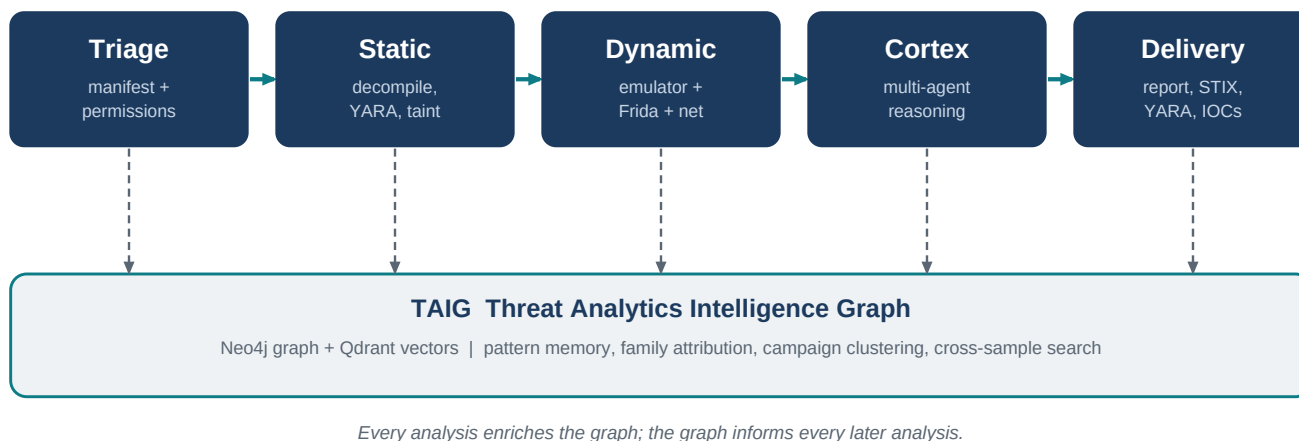


Figure 1. End-to-end analysis pipeline and the compounding knowledge graph.

Triage runs a fast model on the manifest and permissions to route the sample. Static analysis decompiles the APK and grounds a code-intent classification in real code. Dynamic analysis executes the sample under instrumentation. The Cortex reasons over the combined evidence. Delivery emits an evidence-first report and machine-readable intelligence. The knowledge graph records every result and feeds context back into future analyses.

Coverage of the problem statement

Every capability called for in the brief maps to a concrete component of the system.

Required capability	How PARALLAX delivers it
Reverse engineering	jadx decompilation feeding the Code Interpreter agent
Malware pattern recognition	YARA rules, family attribution, and pattern memory in the knowledge graph
Automated code interpretation	Code Interpreter agent classifies decompiled code intent, grounded in real code
Intelligent threat summarization	Synthesis agent produces an evidence-first report and verdict
Static analysis of permissions, APIs, embedded code	androguard, FlowDroid taint (source-to-sink API flows), jadx, YARA
Dynamic analysis of runtime activity and network	Frida hooks on the live process and mitmproxy traffic capture
Severity classification and risk score	Deterministic two-layer score with a LOW to CRITICAL verdict
Investigation report with actionable recommendations	PDF report, fraud-chain reconstruction, prioritized actions, STIX, YARA, IOCs
Faster than manual analysis, built for banks	Automated pipeline at about 0.16 US dollars per sample, bank-readable outputs

3 The Reasoning Cortex

The Cortex is a multi-agent system organized as a directed graph. It runs after static and dynamic stages have produced raw evidence, interprets that evidence, and produces a final verdict with a full evidence trail. Models are matched to task difficulty, which keeps quality high where it matters and cost low everywhere else.

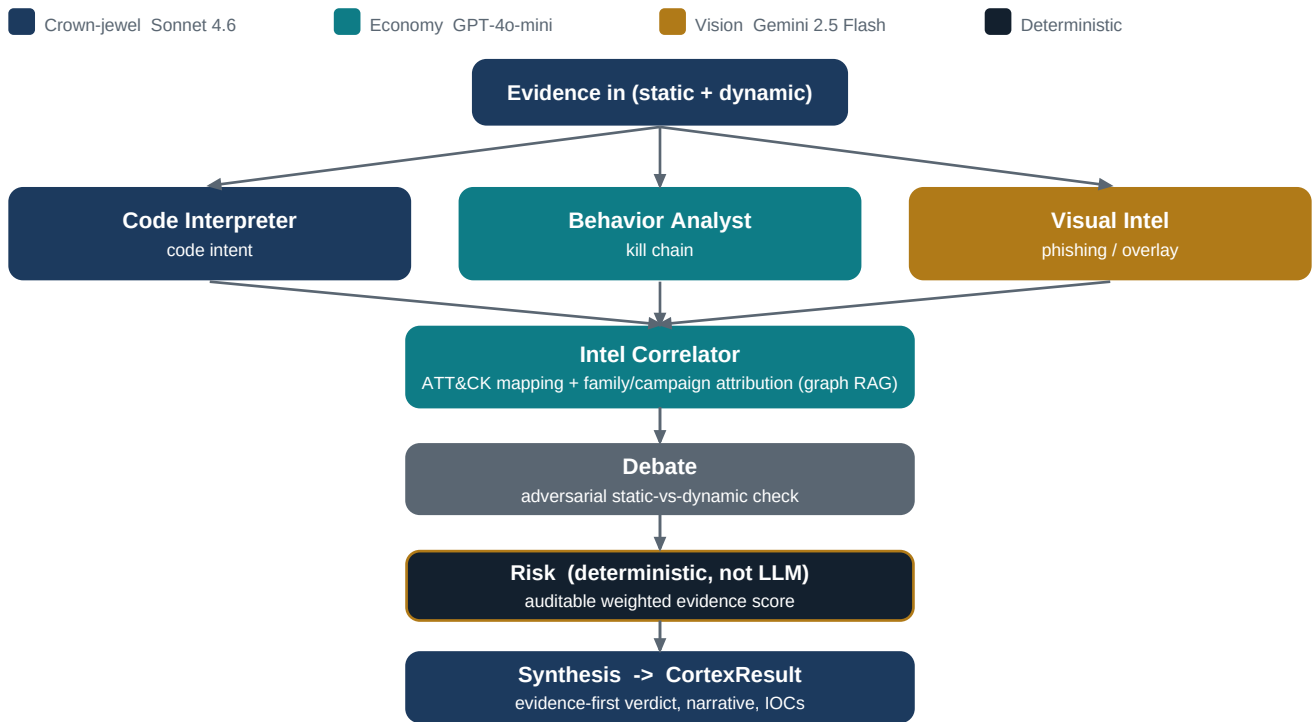


Figure 2. The multi-agent reasoning graph, with models tiered by task.

Agent roster

Agent	Role	Model tier
Code Interpreter	Classifies decompiled code intent (banking trojan, dropper, stealer, adware, benign)	Crown-jewel
Behavior Analyst	Reconstructs the kill-chain from runtime observations	Economy
Visual Intel	Detects overlay attacks and brand impersonation in screenshots	Vision
Intel Correlator	ATT&CK; mapping and family / campaign attribution via graph RAG	Economy + RAG
Debate	Adversarial check for static-versus-dynamic contradiction	Economy
Risk	Two-layer weighted evidence score	Deterministic
Synthesis	Evidence-first report and final verdict	Crown-jewel

Why a multi-agent design, and why the score is deterministic

A single large prompt would be slow, brittle, hard to audit, and expensive. Decomposition lets the independent agents run in parallel, isolates failures to one agent, makes every step a structured object persisted with a confidence value, and lets each task use the right model. Crucially, the final score is not an LLM output. It is a weighted sum of evidence categories, each scored from zero to one and clamped to a 0 to 100 scale, where every component links back to the evidence that produced it. A reviewer can audit any verdict by walking that chain.

Two-layer risk score (deterministic)

Risk component	Weight	Evidence
Runtime observations	0.30	Frida hooks on sensitive APIs, captured C2 traffic
Code intent	0.20	Classification of decompiled code
Visual phishing	0.15	Overlay and brand-impersonation detection

Risk component	Weight	Evidence
ATT&CK; coverage	0.15	Distinct Mobile ATT&CK; techniques matched
Packer / obfuscation	0.10	APKiD packer and protector signatures
Family attribution	0.05	Confirmed known family from threat intel
Other signals	0.05	Manifest and certificate anomalies

When a confirmed known family is detected, for example Cerberus via a MalwareBazaar hash lookup, an auditable HIGH floor is applied: the verdict will not drop below malicious regardless of the narrative. This guards against model under-confidence on fresh-looking variants of known families.

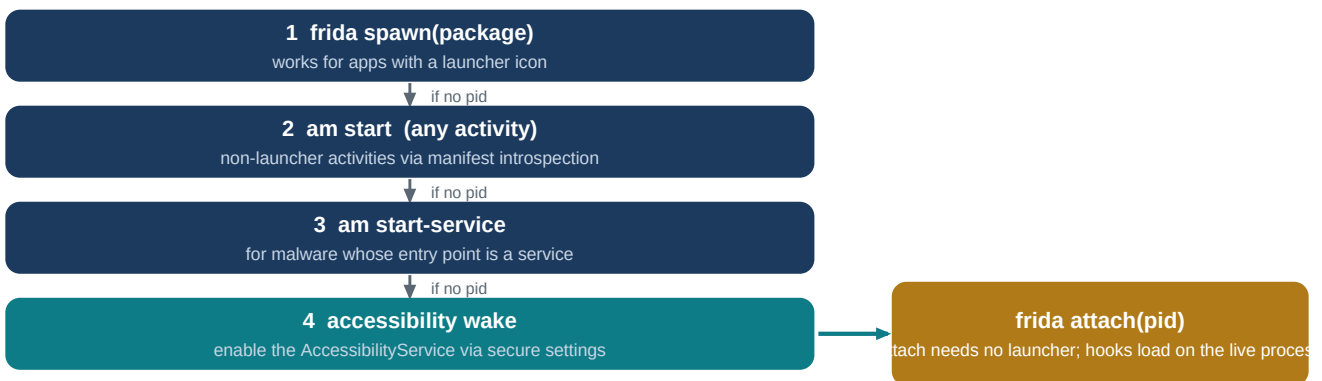
4 The Evidence Pipeline: Static and Dynamic

Static analysis

From the raw APK, the static stage produces a decompiled source tree, manifest and certificate data, permission list, packer signatures, YARA matches, and a source-to-sink taint map. The Code Interpreter reads a ranked subset of the decompiled code, top files by sensitive-API signal with framework packages filtered out, so the classification is grounded in real code rather than manifest metadata alone.

Dynamic analysis and the launch-resistance problem

The hard problem in dynamic analysis of modern Android malware is launch resistance. Trojans hide the launcher icon, place the payload in a service with no main activity, and obfuscate the manifest. A naive call to spawn the launcher fails. PARALLAX uses a launch fallback chain and, once any process is running, attaches to it rather than spawning it, since attach needs no launcher.



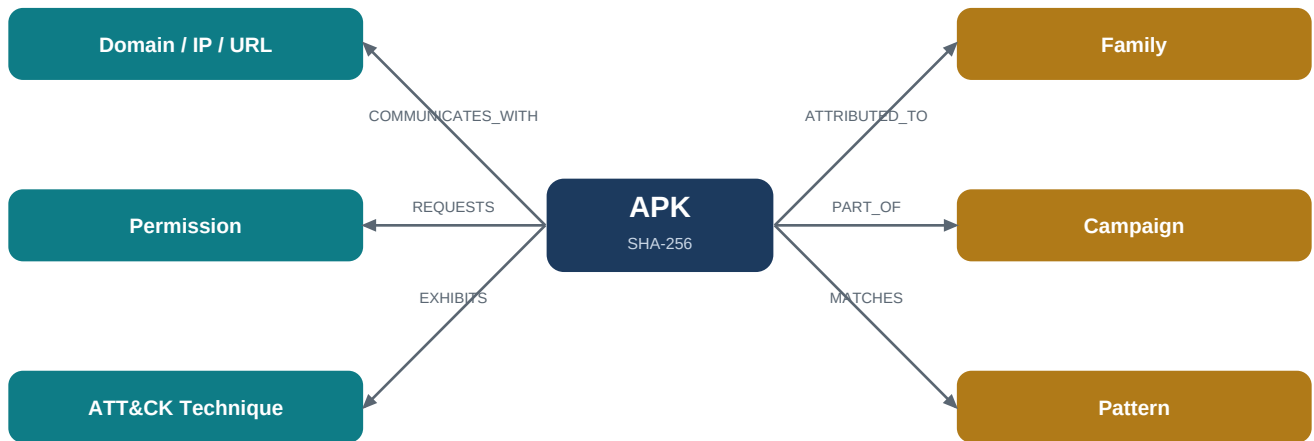
The launch strategy used is recorded on every run, so a result is always provably instrumented.

Figure 3. Launch fallback chain for icon-hiding malware.

Once a process is reached, the system injects pre-built Frida Java hooks on high-signal APIs (SMS interception, accessibility hijacking, crypto key extraction, content-provider theft), routes device traffic through mitmproxy to capture HTTP and TLS, and drives the UI with DroidBot while capturing screenshots. Hook callbacks, network flows, and screenshots are aggregated into structured observation records. A prelude beacon records that instrumentation attached, so a run with zero malicious observations is distinguishable from a run that never instrumented the sample.

5 The Knowledge Graph: A Compounding Layer

Every analysis enriches a shared Threat Analytics Intelligence Graph, stored in Neo4j for structure and Qdrant for semantic vectors. The graph is what makes PARALLAX a learning system rather than a per-analysis tool.



All writes use MERGE semantics, so re-analysis updates rather than duplicates.

Figure 4. Core node and relationship types in the knowledge graph.

After a hundred or more samples, the graph answers questions no single-analysis tool can: which other samples talk to a given C2, what a new variant of a family looks like by semantic similarity, and whether a new campaign is forming via community detection over shared infrastructure. The Intel Correlator queries the graph during analysis, so a report can state that a sample shares a C2 with prior samples attributed to a named campaign. That is the difference between a per-analysis report and an intelligence product.

6 Datasets

PARALLAX operates over real-world, labeled, observable Android malware, not curated toy examples. Datasets fall into four roles: malware sample corpora for analysis and validation, benign controls to measure false positives, threat-intelligence feeds for attribution and standards mapping, and a reference corpus for visual brand comparison.

Primary data sources

Source	What it provides	Access
MalwareBazaar (abuse.ch)	Labeled samples with family tags, hashes, signatures, malware type	Public API and bulk download; hash lookup needs no key
AndroZoo (Univ. of Liege)	20M+ APKs, malicious and benign, with VirusTotal labels	Free for academic use, on request
VirusTotal	Per-hash family attribution, behavior tags, detection counts	API, lookup
MITRE ATT&CK; Mobile	Standardized technique catalog for mobile malware	Public, free (STIX 2.1)
MISP and abuse.ch feeds	Indicators, events, and campaign context (ThreatFox, URLhaus)	Community feeds or self-hosted
Internal TAIG corpus	Samples already analyzed by PARALLAX; grows per submission	Native to the system

Sample coverage

Malicious families targeted in priority order, based on current banking-trojan intelligence: Cerberus (accessibility overlay, hidden icon), Hydra (modular dropper), SharkBot (automatic transfer system, anti-analysis), Anatsa and TeaBot (accessibility based), SpyNote (wide RAT capability), Octo and Coper (C2-heavy), FluBot (SMS worm), BRATA, and Joker (billing fraud). Benign controls include popular Google Play apps across banking, communication, and utility categories, open-source reference apps, and purpose-built apps with synthetic suspicious behavior for end-to-end testing.

Validation methodology

For every submitted sample the system maintains a validation record, so a run on one sample is a case study and a table of 200 or more samples is a validated system. The target corpus is 50 or more samples each from four primary families, 30 or more each from two or three additional families, and 20 or more benign controls. Risk-score calibration is trained and validated against this corpus.

Field	Source of ground truth
True family	MalwareBazaar, VirusTotal, AndroZoo labels
True verdict (clean / suspicious / malicious)	Threat-intel consensus, two or more sources
System verdict, family, risk score	PARALLAX output, compared against truth
Per-stage latency and cost	Telemetry (tokens times model rate)

Data governance: all samples are handled as live malware. Files are quarantined in object storage, are never executed on the API host, and run only inside isolated emulators whose outbound traffic is confined to the capture proxy. Cloud model routing can be disabled entirely for on-premise, data-residency deployments.

7 Open-Source Technology Stack

PARALLAX is built on established open-source tools. Generative-AI calls are routed through a single gateway abstraction that allows fully local operation when no cloud keys are configured.

Component	Purpose	License
jadx	Dex-to-Java decompilation	Apache 2.0
androguard	Manifest, certificate, and DEX analysis	Apache 2.0
YARA	Pattern matching over decompiled code	BSD-2
APKiD	Packer and obfuscator identification	MIT
FlowDroid	Static taint analysis (sources to sinks)	GPL-2.0
Frida	Dynamic Java instrumentation and hooks	wxWindows
mitmproxy	TLS-intercepting proxy for network capture	MIT
DroidBot	UI automation driver for dynamic runs	MIT
Android emulator (QEMU)	Instrumented x86_64 execution environment	Apache 2.0
Neo4j	Threat-intelligence knowledge graph	GPL-3.0 (community)
Qdrant	Vector search across submissions	Apache 2.0
FastAPI, Celery, Redis	API, distributed task queue, broker	MIT / BSD
PostgreSQL, MinIO	Relational metadata, object storage	PostgreSQL / AGPL-3.0
MISP, stix2	Threat-intel sharing and STIX 2.1 export	AGPL-3.0 / BSD
Ollama	Local model serving (fully on-premise mode)	MIT

8 Deliverables and Early Validation

Produced for every analysis

- **Evidence-first PDF report** readable by a bank security lead: executive summary, evidence table, ATT&CK; mapping, IOC list, a ten-stage fraud-chain reconstruction, and prioritized, actionable recommendations.
- **STIX 2.1 bundle** compatible with MISP, OpenCTI, and any TAXII consumer.
- **Auto-generated YARA rule**, compile-validated before delivery, plus network signatures derived from captured traffic.
- **Signed webhooks** to SIEM and SOAR endpoints, and knowledge-graph updates.

Early validation

The current build runs the full pipeline end-to-end. On a live Cerberus banking-trojan sample sourced from MalwareBazaar, the system launched the icon-hidden payload through the accessibility-wake path, attached the instrumentation, captured a runtime observation, attributed the family as Cerberus, and returned a HIGH verdict with a complete evidence trail, at a measured cost in the range of 0.15 to 0.20 US dollars per sample. The next milestone is the 200-sample validation corpus and the empirical calibration layer trained on it.